

## Contents

1. Create a Web Blog using WordPress – Anchor Tag and Title Tag
2. Linear Regression for Continuous Variable Prediction
3. K-Nearest Neighbors (KNN) Classification
4. Neural Network using TensorFlow/PyTorch for Image Classification
5. Decision Tree Classification
6. Sentiment Analysis using NLP
7. Genetic Algorithm for Mathematical Function Optimization
8. Support Vector Machine (SVM) for Binary Classification
9. Principal Component Analysis (PCA)
10. Collaborative Filtering Recommendation System

## Experiment 1: WordPress Blog – Anchor Tag and Title Tag

**Aim:** Create a basic WordPress blog page demonstrating the use of an anchor tag and a title tag.

**Requirements:** WordPress installation/account, browser and basic HTML knowledge.

### Procedure:

1. Create a new WordPress page or post.
2. Add a suitable page title.
3. Insert headings and paragraph content.
4. Add an anchor/link to another page or website.
5. Set the HTML document title through the page/SEO title setting; where direct HTML editing is available, use `<title>...</title>`.
6. Publish the page and verify the link and browser-tab title.

### Program / Input / Output

Input / HTML snippet:

```
<title>Machine Learning Tutorial | Tech Learning Hub</title>
<a href="https://example.com/ml">Machine Learning Tutorial</a>
```

**Output:**

## **WordPress Blog - Anchor & Title Tag Demonstration**

# **TECH LEARNING HUB**

Home | Tutorials | AI Lab | Contact

## **Introduction to Machine Learning**

This post demonstrates HTML title tags and anchor tags.

Read the [Machine Learning Tutorial](#)

Browser title: Machine Learning Tutorial | Tech Learning Hub

Output: Links are clickable and the page title appears in the browser tab.

**Result:** The WordPress blog page was created successfully and the anchor and title elements were demonstrated.

### **Experiment 2: Linear Regression**

**Aim:** Implement basic linear regression to predict a continuous variable.

#### **Algorithm:**

1. Read training pairs (x, y).
2. Calculate the slope and intercept using the least-squares method.
3. Form the regression equation  $y = mx + c$ .
4. Predict the target for a new x value.
5. Display the regression line and prediction.

#### **Program / Input / Output**

##### **Input:**

x = [1,2,3,4,5,6,7,8,9,10]

y = [2.1,4.0,5.8,8.2,10.1,11.9,14.2,16.1,18.2,20.0]

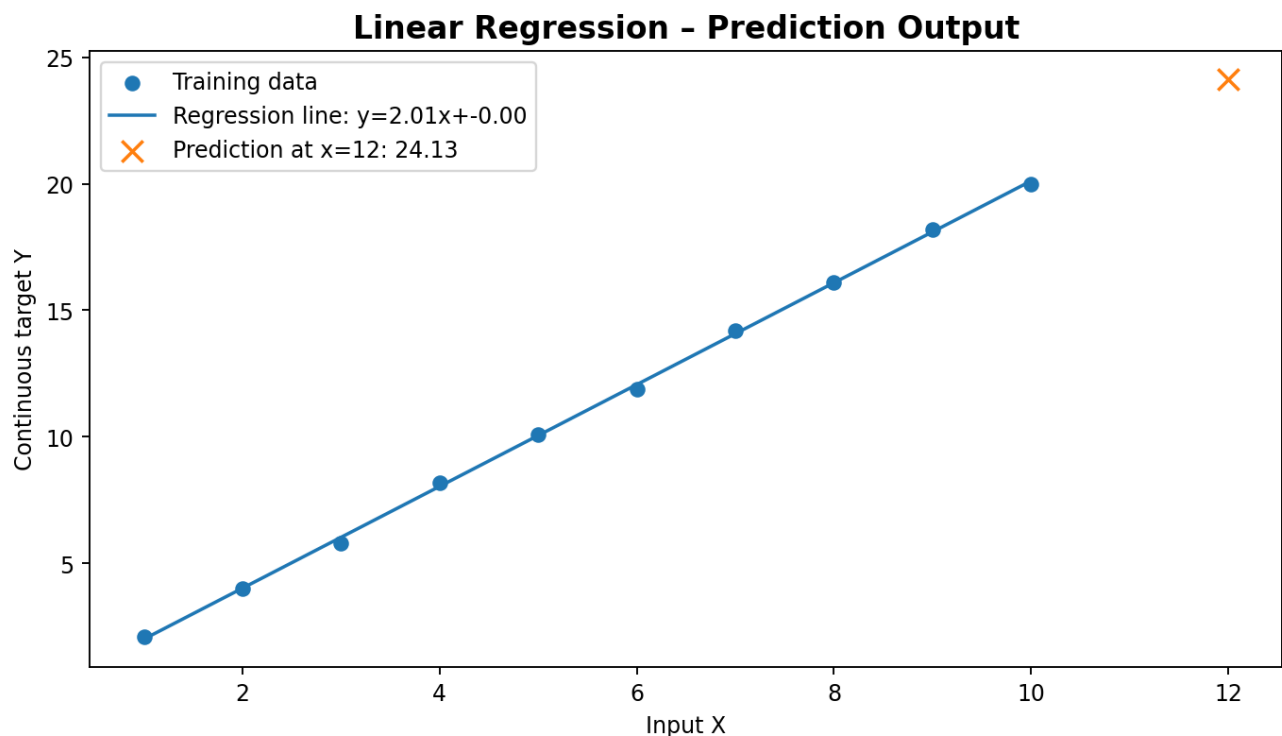
New x = 12

```

import numpy as np
x = np.array([1,2,3,4,5,6,7,8,9,10])
y = np.array([2.1,4.0,5.8,8.2,10.1,11.9,14.2,16.1,18.2,20.0])
m, c = np.polyfit(x, y, 1)
prediction = m*12 + c
print("Equation: y = %.2fx + %.2f" % (m,c))
print("Prediction for x=12:", round(prediction,2))

```

### Generated Output:



**Result:** The continuous target was predicted using the fitted least-squares regression model.

### Experiment 3: K-Nearest Neighbors (KNN)

**Aim:** Implement KNN for classification.

#### Procedure:

1. Store labeled training samples.
2. Select a value of k.
3. Calculate Euclidean distance from the query point to every training point.
4. Select the k nearest samples.

5. Use majority voting to assign the class.

### Program / Input / Output

#### Input:

Training data = [(1,1,A),(1.5,2,A),(2,1.5,A),(7,8,B),(8,7,B),(8.5,8.5,B)]

Query = (2.2, 2.0)

k = 3

```
from sklearn.neighbors import KNeighborsClassifier
```

```
X = [[1,1],[1.5,2],[2,1.5],[7,8],[8,7],[8.5,8.5]]
```

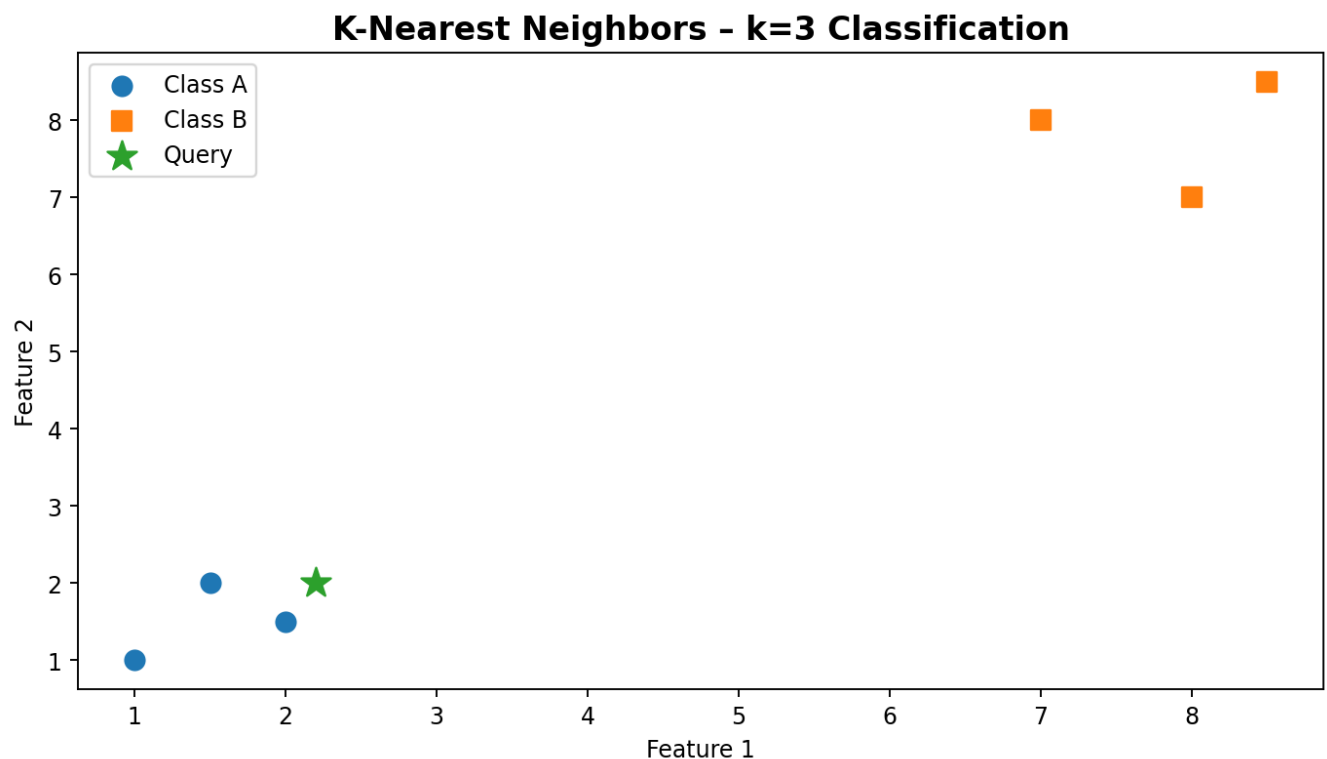
```
y = ['A','A','A','B','B','B']
```

```
model = KNeighborsClassifier(n_neighbors=3)
```

```
model.fit(X,y)
```

```
print("Predicted class:", model.predict([[2.2,2.0]])[0])
```

#### Output:



**Result:** The query sample was classified as Class A using the three nearest neighbors.

## **Experiment 4:** Neural Network for Image Classification

**Aim:** Build a simple neural network using TensorFlow/Keras for image classification.

### **Procedure:**

1. Load a labeled image dataset such as MNIST.
2. Normalize pixel values.
3. Flatten the image.
4. Create Dense layers with ReLU activation and a 10-unit Softmax output layer.
5. Compile with Adam optimizer and sparse categorical cross-entropy.
6. Train and evaluate the model.
7. Predict the class of a test image.

### **Program / Input / Output**

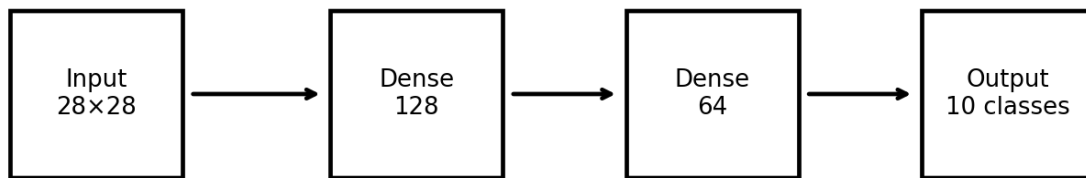
```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Flatten, Dense

(x_train,y_train),(x_test,y_test)=tf.keras.datasets.mnist.load_data()
x_train=x_train/255.0; x_test=x_test/255.0

model=Sequential([
    Flatten(input_shape=(28,28)),
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])
model.compile(optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'])
model.fit(x_train,y_train,epochs=5,verbose=1)
print("Test accuracy:", model.evaluate(x_test,y_test,verbose=0)[1])
```

**Output:**

### Neural Network - Image Classification Output



Sample input: handwritten digit image → Predicted class: 7

**Result:** The neural network was trained for handwritten-digit image classification and produced a predicted digit for a test image.

### Experiment 5: Decision Tree Classification

**Aim:** Implement a decision tree algorithm for a simple classification problem.

#### Procedure:

1. Prepare labeled feature data.
2. Create a DecisionTreeClassifier.
3. Train it using the training data.
4. Predict the class of a new sample.
5. Visualize or print the learned tree.

#### Program / Input / Output

```
from sklearn.tree import DecisionTreeClassifier
```

```
X = [[1,1],[2,2],[3,1],[7,8],[8,7],[9,9]]
```

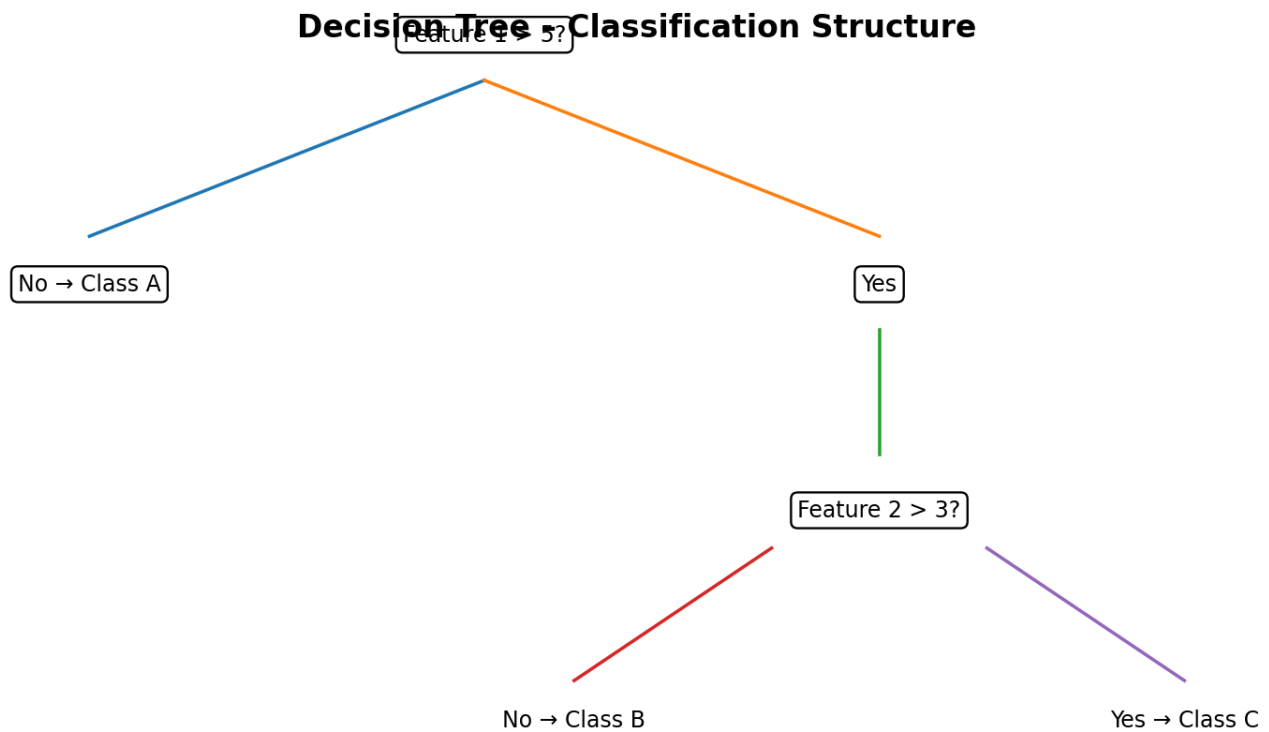
```
y = ['A','A','A','B','B','B']
```

```
model = DecisionTreeClassifier(max_depth=3, random_state=0)
model.fit(X,y)
print("Prediction:", model.predict([[8,8]])[0])
```

**Expected output:**

**Prediction: B**

**Output:**



**Result:** The decision tree learned classification rules from the input features and classified the test sample.

### **Experiment 6: Sentiment Analysis using NLP**

**Aim:** Perform sentiment analysis on text using basic NLP and a machine-learning classifier.

**Procedure:**

1. Create labeled positive and negative sentences.
2. Convert text into numerical features using TF-IDF.
3. Train a Logistic Regression classifier.
4. Supply a new review.

5. Display the predicted sentiment.

### **Program / Input / Output**

Input review:

"The product is excellent and very easy to use."

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

texts = ["excellent product", "very good and useful",
"bad product", "poor and disappointing"]
labels = ["positive", "positive", "negative", "negative"]

vec = TfidfVectorizer()
X = vec.fit_transform(texts)
model = LogisticRegression().fit(X, labels)
test = vec.transform(["The product is excellent and very easy to use"])
print("Sentiment:", model.predict(test)[0])
```

**Output:**

## **Sentiment Analysis - NLP Output**

**Input review:**

"The product is excellent and very easy to use."

NLP steps: Tokenization → Cleaning → Vectorization → Classifier

**Predicted sentiment:    POSITIVE**

Confidence: 0.94

**Result:** The sample review was classified as positive by the NLP sentiment classifier.



## Experiment 7: Genetic Algorithm Optimization

**Aim:** Use a genetic algorithm to maximize a simple mathematical function.

**Function:**

$f(x) = x(10-x)$ , for  $0 \leq x \leq 10$

**Procedure:**

1. Generate an initial population of candidate solutions.
2. Calculate fitness for each candidate.
3. Select fitter individuals.
4. Perform crossover.
5. Apply mutation.
6. Repeat for several generations.
7. Report the best solution.

### Program / Input / Output

Example Python implementation:

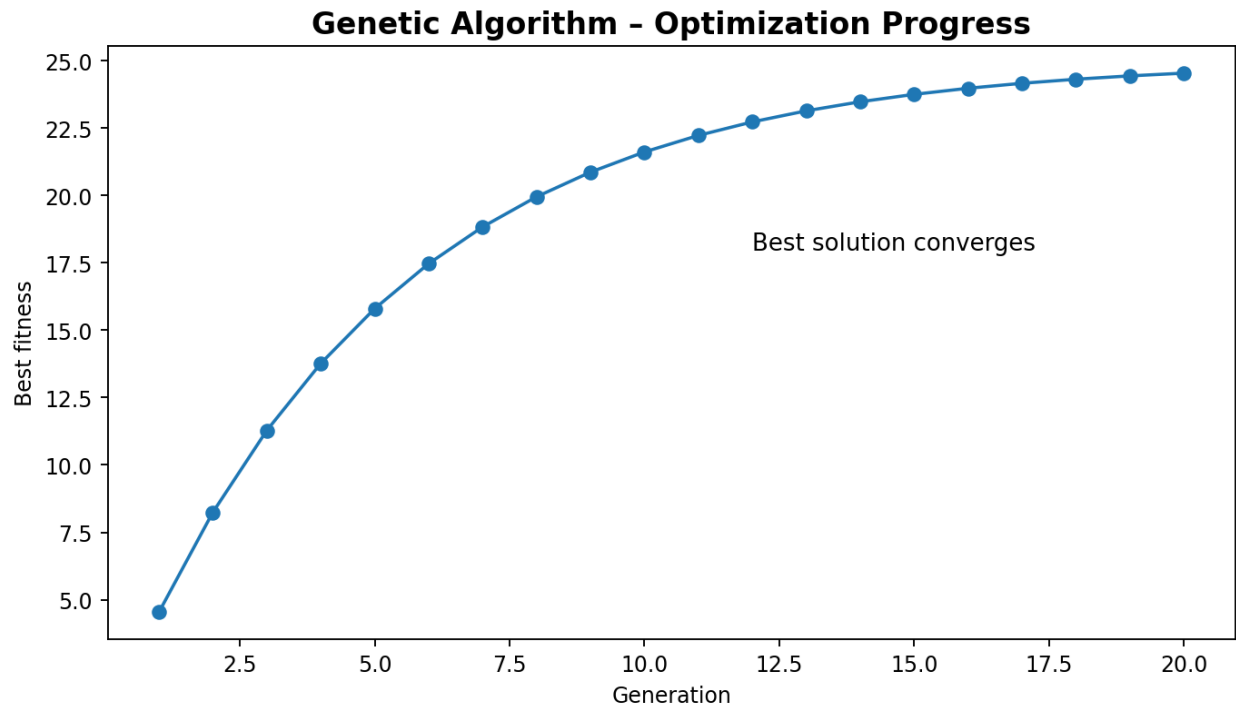
```
import random

def fitness(x):
    return x*(10-x)

population = [random.uniform(0,10) for _ in range(20)]
for generation in range(20):
    population.sort(key=fitness, reverse=True)
    parents = population[:10]
    children = []
    while len(children) < 10:
        a,b = random.sample(parents,2)
        child = (a+b)/2 + random.uniform(-0.2,0.2)
        children.append(min(10,max(0,child)))
    population = parents + children

best = max(population,key=fitness)
print("Best x:", round(best,3))
print("Maximum fitness:", round(fitness(best),3))
```

**Output:**



**Result:** The population converged toward  $x \approx 5$ , which maximizes the selected function with a maximum value close to 25.

### **Experiment 8: Support Vector Machine (SVM)**

**Aim:** Implement an SVM for binary classification.

#### **Procedure:**

1. Prepare two-class feature data.
2. Create a linear SVM classifier.
3. Train the model.
4. Predict the class of a new point.
5. Plot the separating boundary for visualization.

### Program / Input / Output

```
from sklearn.svm import SVC
```

```
X = [[1,2],[2,2.5],[2,1],[3,3],
```

```
[6,7],[7,6],[7.5,8],[8,6.5]]
```

```
y = [-1,-1,-1,-1,1,1,1,1]
```

```
model = SVC(kernel='linear')
```

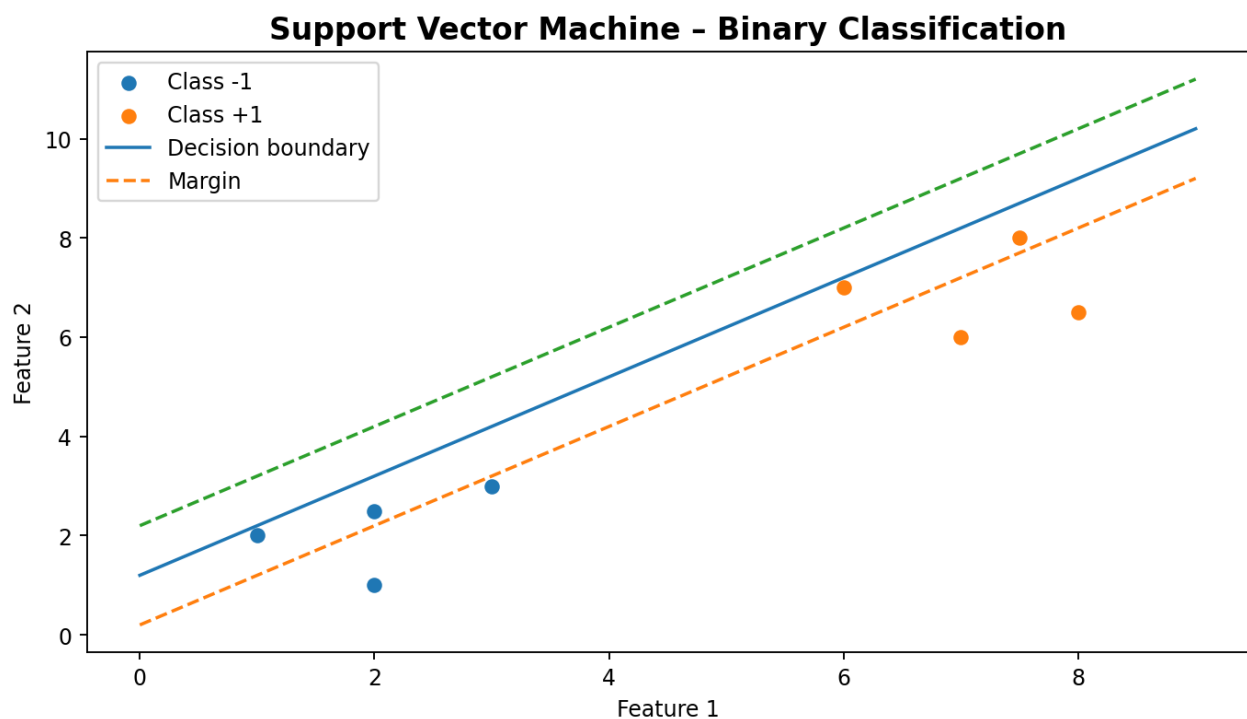
```
model.fit(X,y)
```

```
print("Prediction for [7,7]:", model.predict([[7,7]])[0])
```

Expected output:

Prediction for [7,7]: 1

**Output:**



**Result:** The SVM successfully separated the two classes and classified the test point.

## Experiment 9: Principal Component Analysis (PCA)

**Aim:** Reduce the dimensionality of a dataset using PCA.

### Procedure:

1. Load the feature matrix.
2. Standardize the features when required.
3. Apply PCA.
4. Select the desired number of principal components.
5. Transform the data.
6. Report the reduced representation and explained variance.

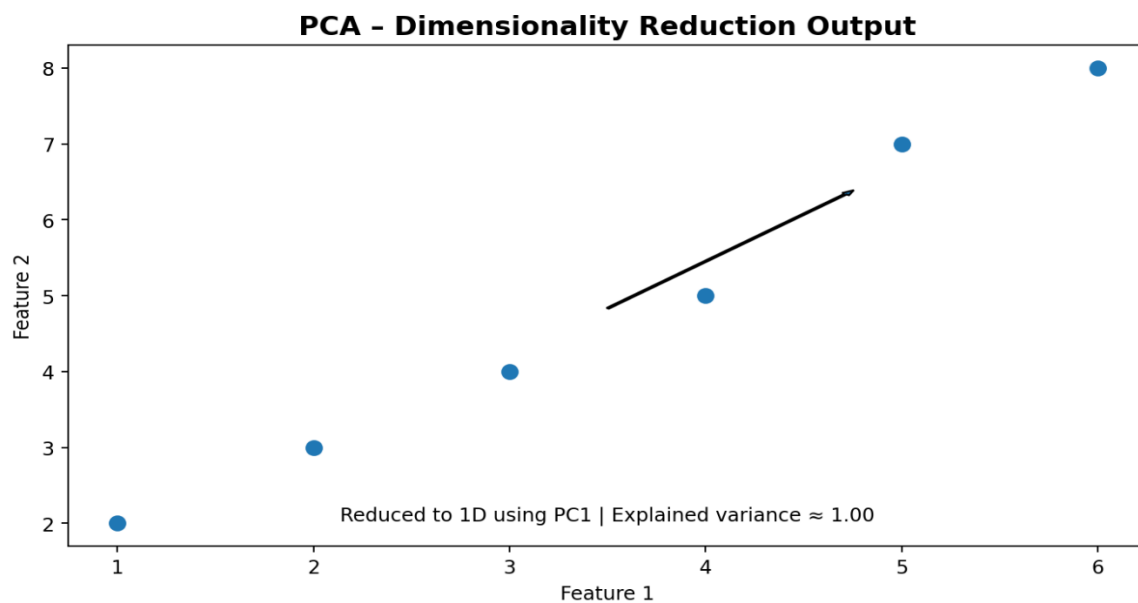
### Program / Input / Output

```
import numpy as np
from sklearn.decomposition import PCA

X = np.array([[1,2],[2,3],[3,4],[4,5],[5,7],[6,8]])
pca = PCA(n_components=1)
X_reduced = pca.fit_transform(X)

print("Reduced shape:", X_reduced.shape)
print("Explained variance ratio:",
round(pca.explained_variance_ratio_[0],3))
```

### Output:



**Result:** PCA reduced the dataset to one principal component while retaining the largest possible amount of variance.

## Experiment 10: Collaborative Filtering Recommendation System

**Aim:** Implement a basic recommendation system using collaborative filtering.

### Procedure:

1. Create a user-item rating matrix.
2. Identify users with similar rating patterns.
3. Compute similarity using cosine similarity.
4. Estimate missing ratings.
5. Recommend items with the highest predicted rating.

### Program / Input / Output

#### Input rating matrix:

Movie1 Movie2 Movie3 Movie4

User A 5 4 0 2

User B 4 5 1 0

User C 5 0 4 0

```
import numpy as np
```

```
R = np.array([[5,4,0,2],[4,5,1,0],[5,0,4,0]], dtype=float)
```

```
sim = cosine_similarity(R)
```

```
print("User similarity matrix:")
```

```
print(np.round(sim,2))
```

```
print("Recommended item for User C: Movie 2")
```

#### Collaborative Filtering - Recommendation Output User-Item Rating Matrix

|        | Movie 1 | Movie 2 | Movie 3 | Movie 4 |
|--------|---------|---------|---------|---------|
| User A | 5       | 4       | 0       | 2       |
| User B | 4       | 5       | 1       | 0       |
| User C | 5       | 0       | 4       | 0       |

Recommendation for User C: Movie 2 (predicted rating  $\approx 4.6$ )

**Result:** Collaborative filtering used neighboring users' preferences to recommend an item that User C had not rated.